

Programming Assignment 10: Graph Algorithms

Due: Thursday, April 23rd at 11pm

Learning Goals

- To implement key graph algorithms and understand their applications
- To understand and analyze a provided codebase
- To evaluate and compare algorithmic rankings with human-determined rankings

Assignment Guidelines and Specifications

This is an individual assignment. All code submitted must be entirely your own.

For this assignment, there are additional requirements:

- You may use any classes and methods from the Java standard library.
- You must implement the required graph algorithms as described below.
- You may (and are encouraged to) create private helper methods to remove redundancy and provide structure to your solution.
- Ensure your IDE does not add non-standard Java imports.
- **Do not alter any methods in any of the files we provided you other than the three methods we ask you to write. You do not need to add anything extra to this assignment. You only need to write the three described methods. You are still free to write private helper methods for these three methods as you deem necessary, but this is the only extra thing you should add.**

Introduction

Prior to the 2014 season, Division I college football did not have an NCAA-run tournament to determine its champion. Instead, the Bowl Championship Series (BCS) used various polls and surveys to select the "best" two teams for the final game of the season. The ranking algorithm the BCS used consisted of two human surveys and six algorithms carried out by computers. Note that these algorithms used by the BCS did not take into account game scores, only whether a team won or lost a game, since the people running the BCS thought it unsportsmanlike to encourage stronger teams to run up the score on weaker teams. From 2017 to 2024, Division I college football used a four-team playoff system, where a human selection committee determined the top four teams. The selection committee claimed to not use any algorithms in determining its rankings, but there were still a large number of individuals and sites that published rankings based on algorithms.

In this assignment, you will complete a `Graph` class and a `FootballRanker` class to rank college football teams based on the graph formed by Division I teams and the games they played against each other. [Centrality of vertices](#) in the graph, with some adjustments, will be used to determine the "best" team. Your program will rank teams in three different ways and compare the results to the [Associated Press \(AP\) end-of-season poll](#), a human-determined ranking system with no algorithmic input.

Your program will compute team rankings using three different approaches:

1. **Unweighted Shortest Path Ranking**
 - Rank each team by calculating the number of other teams they are connected to.
 - For each connected team, calculate the shortest **unweighted** path (i.e., the number of edges in the path).
 - Compute the average **unweighted** path length by dividing the sum of all shortest unweighted path lengths by the number of connected teams (vertices).
2. **Weighted Shortest Path Ranking**
 - Rank each team by calculating the number of other teams they are connected to.
 - For each connected team, calculate the shortest **weighted** path (i.e., the path length considering edge weights).
 - Compute the average **weighted** path length by dividing the sum of all shortest weighted path lengths by the number of connected teams (vertices).
3. **Weighted Path Adjusted by Win/Loss Percentage**
 - Use the ranking from Method 2 (Weighted Shortest Path Ranking).
 - Adjust the ranking by dividing the average weighted path length by the team's win/loss percentage.

Most of the program is already done. There are only three methods you must implement:

1. **dijkstra in Graph.java**: Compute the shortest weighted path from a given start vertex to all other vertices.
2. **findAllPaths in Graph.java**: Compute connectivity information for each vertex, including shortest unweighted and weighted paths and the number of other vertices this vertex is connected to
3. **printRootMeanSquareError in FootballRanker.java**: Compare graph-based rankings to AP poll rankings for football teams.

Getting Started

Here are the necessary files for this assignment:

- [**Graph.java**](#): This file implements a Graph data structure that supports basic graph operations such as pathfinding and traversal. You will need to complete the methods `dijkstra` and `findAllPaths` in this file to compute the shortest paths. These methods will be used in the `FootballRanker` class to calculate football team rankings. You must submit this file once completed.
- [**FootballRanker.java**](#): This is the main class where the ranking of football teams happens. It uses the `Graph` class to process game results and calculate rankings. The rankings are then compared to the human-generated rankings, such as the Associated Press poll. You will need to implement the `printRootMeanSquareError` method to compare the algorithmic rankings with the AP poll. You must submit this file once completed.

- [GraphAndRankTester.java](#): This file contains test cases to check the correctness of the three methods you must implement. You will need to add additional tests as described below. There are also two questions (described below) that you must answer at the top of the file. You must submit this file once completed.
- [FootballRecord.java](#): This class models the win/loss record of a football team. This file is provided for you and does not need to be modified or submitted.
- [AllPathsInfo.java](#): This class contains information about all paths from a given start vertex in a graph to all other vertices. It is used by the Graph class to support graph traversal and pathfinding operations. This file is provided for you and does not need to be modified or submitted.
- [Documentation](#): This is documentation for the provided classes above. You should refer to this extensively.
- **Testing Files:** The following are data files for you to use when testing your program. The files labeled `div120XX.txt` contains the results of all Division I college football games from the year 20XX. The files labeled `20XXap_poll.txt` contain the final Division I Associated Press rankings for the year 20XX. After running your program on these tester files, you may post your results on Ed Discussion to compare with other students in the class.
 - [2005ap_poll.txt](#) and [div12005.txt](#)
 - [2008ap_poll.txt](#) and [div12008.txt](#)
 - [2014ap_poll.txt](#) and [div12014.txt](#)
- [games08.txt](#): This file contains all college football games from 2008, not just the Division I teams. You will use this file when answering questions later in this assignment.
- [GraphSampleOutput.txt](#): This is the expected output for your program when using the 2008 data files. Your output must match this exactly.

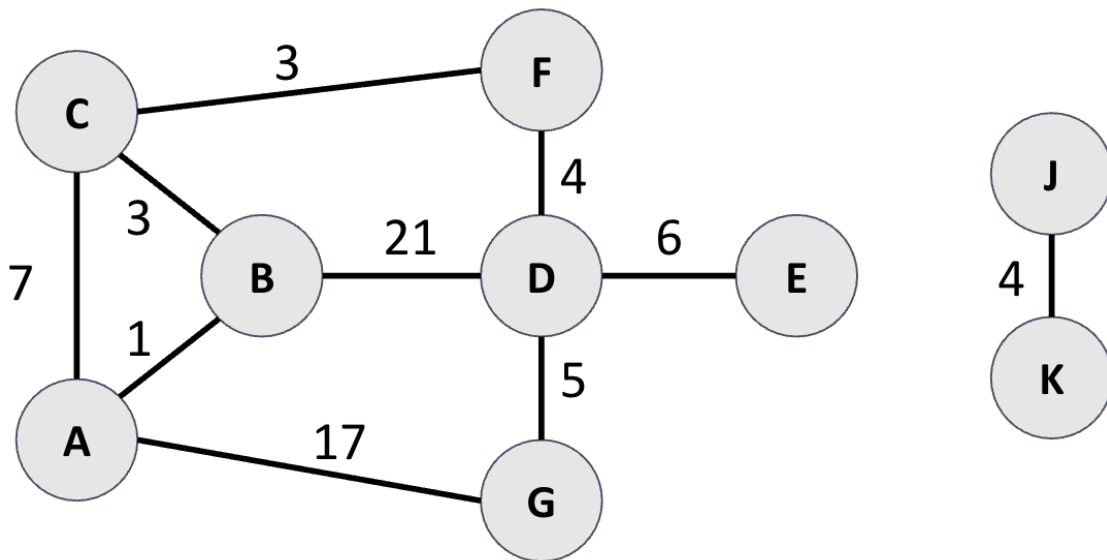
Graph Class

1. You must implement the `dijkstra` method in the Graph class. This method finds the shortest weighted path from a given start vertex to all other vertices in the graph using Dijkstra's Shortest Weighted Path Algorithm. Read the method documentation thoroughly to understand all the requirements of the method.
2. You must implement the `findAllPaths` method in the Graph class. This method updates each vertex in the Graph so that it stores the number of other vertices it is connected to (referred to as the *number of paths* from the vertex in this assignment). This method also calculates the sum of all the shortest weighted and unweighted paths from the vertex to every other vertex it connects to. If the boolean parameter `weighted` is `false`, these values will be the same. Each `Vertex` object has instance variables to store this data, including `numVertexConnected`, `totalUnweightedPathLength`, and `totalWeightedPathLength`. Read the comments for these variables in the `Vertex` class to help clarify their purpose and how they should be updated in this method.

The method must call either `findUnweightedShortestPaths` or `dijkstra` as necessary. The `Vertex` class, the nested `Path` class, and the `getPath(String)` method are useful for completing and testing this method. While the majority of the support code is provided, determining how to use it effectively is necessary. Additional helper methods may be written if needed, but the output must match the expected results.

The method should also identify and store the "longest shortest path" in the graph by tracking the highest-cost shortest path found. Since it computes the shortest paths between all pairs of vertices, it keeps a record of the longest of these paths based on weighted cost. When using `findUnweightedShortestPaths`, this cost is equivalent to the number of edges in the path. The `getPath` method should be used to create a `Path` object for the longest path found, which is then stored in the instance variable `longest`.

Consider the following example of a weighted, undirected graph:



If we execute the `findAllPaths` method with the `weighted` parameter set to `false`, the cumulative sum statistics in the vertex objects should be as follows:

Vertex	numVertexConnected	totalUnweightedPathLength	totalWeightedPathLength
A	6	10.0	10.0
B	6	9.0	9.0
C	6	10.0	10.0
D	6	8.0	8.0
E	6	13.0	13.0
F	6	10.0	10.0
G	6	10.0	10.0
J	1	1.0	1.0
K	1	1.0	1.0

If we execute the `findAllPaths` method with the `weighted` parameter set to `true`, the cumulative sum statistics in the vertex objects would be as follows:

Vertex	numVertexConnected	totalUnweightedPathLength	totalWeightedPathLength
A	6	20.0	56.0
B	6	15.0	51.0
C	6	12.0	42.0
D	6	12.0	43.0
E	6	17.0	73.0
F	6	11.0	39.0
G	6	17.0	68.0
J	1	1.0	4.0
K	1	1.0	4.0

FootballRanker Class

The `FootballRanker` class computes rankings for teams based on the three approaches described in the "Introduction" section of this assignment handout. This method for this class relies on the `findAllPaths` method, which in turn relies on the `dijkstra` method, so both must be correctly implemented before proceeding.

In this class, each team is a vertex. A directed edge exists between team A and B if team A beat team B in a game. The weight of the edge is determined by the score of the game between teams A and B. The closer the score, the larger the weight.

You must implement the `printRootMeanSquareError` method in the `FootballRanker` class. The `printRootMeanSquareError` method compares the computed rankings from `findAllPaths` with the Associated Press (AP) end-of-season poll rankings and calculates the root mean square error (RMSE) between them. The AP rankings are provided as a `List` parameter where the index represents the rank (using zero-based indexing), while the computed/graph rankings are stored in a `TreeSet` parameter. Iterate through the AP ranks and determine the root mean square error between the AP ranks and the calculated ranks. The root mean square error is determined by taking the difference between the predicted graph rank and the AP rank, squaring the difference, adding all of these squared differences together, dividing by the number of teams, and taking the square root.

You can calculate the root mean square error (RMSE) using the following formula:

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

Here, $\hat{y}_i$ is the predicted rank for the i^{th} team based on the computed rankings from the graph, y_i is the actual rank for the i^{th} team from the AP poll, and n is the total number of teams. **If a team appears in the AP poll rankings but is missing from the computed/graph rankings, assign it a rank equal to one more than the total number of teams in the computed/graph rankings.** The `FootballRanker` class cuts some teams from the predictions if they do not have enough direct or transitive wins (wins based on connections in the graph). If the parameter `showResults` is `true` then the method should print out the data as shown in the given sample output. This method must return the root mean square error rounded to the nearest tenth.

Testing

You are required to write and submit your own test cases in `GraphAndRankTester.java`. You are expected to write:

- 2 test cases for `dijkstra`
- 4 test cases for `findAllPathsInfo`, specifically:
 - 2 tests in which the parameter, `weighted`, is `true`
 - 2 tests in which the parameter, `weighted`, is `false`

In previous assignment guides, we shared a few notes with you about writing test cases. Those notes are copied here for your convenience:

- You may copy the structure of the provided tests.
- Any test cases that you write should not violate the preconditions of the method.
- You must group the test cases by method and comment which method the test case is for.
- You are not required to write test cases for any private helper methods or constructors.
- **Delete the test cases that we have provided for you.**

Questions

There are two questions at the top of `GraphAndRankTester.java` that you must answer. In order to answer these questions, you should run your program on the `games08.txt` file linked above. This file contains all college football games from 2008, not just the Division I teams. Compare the results to those obtained using only Division I teams (some surprises may occur!).

Here are the questions:

1. The assignment presents three ways to rank teams using graphs. The results, especially for the last two methods, are reasonable. However, if all results from all college football teams are included some unexpected results occur. Explain the unexpected results.
2. Suggest another way/method of ranking teams using the results from the graph. Thoroughly explain your method. The method can build on one of the three existing algorithms.

Be thoughtful with your answers. If you are not familiar with the structure of college football in the US, you may need to do some [research](#). Understanding the [divisions within college football](#) can help you provide a more informed response. If you are struggling to understand these concepts, please feel free to attend help hours for assistance. **Answers that demonstrate minimal effort or thought will receive little to no credit.**

Submission Checklist

Before you submit, run through our submission checklist to make sure you didn't forget anything! Did you remember to:

1. Implement all the required methods?
2. Ensure your program passes the given tests? Note: passing the provided tests does not mean your solution is correct. We will use hidden tests to test edge cases.
3. Ensure your program matches the sample output exactly?
4. Add your own tests to `GraphAndRankTester.java` for `dijkstra` and `findAllPaths`, and delete the provided tests?
5. Answer the two required questions in `GraphAndRankTester.java`?
6. Fill in the header comments in `Graph.java`, `FootballRanker.java`, and `GraphAndRankTester.java`?
7. Follow the [program hygiene guide](#)?
8. Follow the assignment specifications from page 3 of the [Assignment handout](#)?

If you have done all of these things, you are ready to submit `Graph.java`, `FootballRanker.java` and `GraphAndRankTester.java` to Gradescope!

Thank you to Professor Mike Scott for creating the previous version of this assignment!